

Herald



Herald — MTProto User-Account Login (Operator Guide)

Field	Value
Revision	1
Created	2026-05-31
Last modified	2026-05-31
Status	active
Status summary	Operator-facing deep-dive for bootstrapping the MTProto user-account session that drives Herald's §11.4.98 fully-autonomous QA flows (<code>qaherald mtproto login</code>). The companion to <code>TELEGRAM.md</code> (which covers the bot side via <code>BotFather</code>): this guide covers the user account side — the real Telegram user the QA harness impersonates because a bot cannot see another bot's group messages (the bot-to-bot wall). Documents WHY a user account vs a bot (§1); the prominent account-safety / ban-risk warnings verified against Telegram's official docs (§2 — VoIP-number ban, recover@telegram.org pre-declaration, cloud 2FA, dedicated QA account); obtaining <code>api_id/api_hash</code> at my.telegram.org/apps step-by-step with the real form-validation gotchas (§3); cloud 2FA + recovery-email setup (§4); the exact five <code>HERALD_MTPROTO_*</code> env vars + <code>.env</code> placement (§5); the <code>qaherald mtproto login</code> interactive walkthrough with the real prompts + session file (§6); <code>whoami verify + logout</code> teardown (§7); running the autonomous live tests E135-E137 + HRD-156 once the session exists (§8); the non-TTY delegated flow <code>HERALD_MTPROTO_ALLOW_NON_TTY=1</code> and when §11.4.98(B) permits it (§9); a 10-entry troubleshooting cookbook keyed on the real Telegram error strings (§10); and the full cross-reference index (§11). Every credential name, command, prompt, and error string is taken verbatim from <code>qaherald/cmd/qaherald/mtproto_cmd.go</code>, <code>qaherald/internal/mtproto/</code>, or Telegram's official documentation — none invented.
Issues	none
Issues summary	—
Fixed	(n/a — new guide)
Continuation	bump when the <code>github.com/gotd/contrib/floodwait + ratelimit</code> middlewares are vendored under <code>submodules/gotd-td/</code> and registered via <code>telegram.Options.Middlewares</code> (today <code>FLOOD_WAIT</code> surfaces as a sanitized error — see §10.5); bump when a local Bot API server lifts the

Field	Value
	attachment caps; bump when the delegated-by-conductor FIFO flow (§9) gains a turnkey script; re-verify the Telegram-source sections at the 90-day staleness boundary (§11 footer).

Table of contents

- [§1. Why an MTProto user-account \(not a bot\)?](#)
 - [§2. Prerequisites + account-safety warnings \(READ FIRST — ban risk\)](#)
 - [§3. Obtain `api\_id` + `api\_hash` at \[my.telegram.org/apps\]\(https://my.telegram.org/apps\)](#)
 - [§4. Set up cloud 2FA + recovery email on the QA account](#)
 - [§5. The env vars + `.env` placement](#)
 - [§6. `qaherald mtpROTO login` — the one-time interactive bootstrap](#)
 - [§7. `whoami` to verify + `logout` to tear down](#)
 - [§8. Running the autonomous live tests \(E135-E137 / HRD-156\)](#)
 - [§9. The non-TTY delegated flow \(`HERALD\_MTPROTO\_ALLOW\_NON\_TTY=1`\)](#)
 - [§10. Troubleshooting cookbook](#)
 - [§11. References](#)
-

§1. Why an MTProto user-account (not a bot)?

Herald's primary messenger integration is a Telegram **bot** (see [TELEGRAM.md](#) for the BotFather walkthrough). A bot is sufficient for production: subscribers DM the bot, the bot relays to Claude Code, the bot replies. But it is NOT sufficient to **fully automate the test of that round-trip** — and that gap is what this guide closes.

§1.1 The bot-to-bot wall

Telegram enforces a structural privacy boundary: **a bot can never see another bot's messages in non-DM contexts**. This is not a permission you can toggle — it is a property of the Bot API.

Empirically verified for Herald on 2026-05-28 (recorded in `docs/qa/HRD-LIVE-MTPROTO-20260528T125321Z/` and the `telegram_bot_to_bot_wall` memory): a second bot, `@pherald_qa_bot` (id 8971749017), sent `message_id=18` to the group -4946584787 ("ATMOSphere Development"); the target worker bot's `getUpdates` poller observed **0 updates**. A QA harness built on a second bot is therefore dead on arrival — it can speak but the system-under-test never hears it.

§1.2 What §11.4.98 demands, and why MTProto answers it

HelixConstitution **§11.4.98 (Full-Automation Anti-Bluff Mandate)** requires every Herald test — unit, integration, e2e, Challenge, stress, chaos, live — to be fully self-driving end-to-end with **no human action during execution**. A test that requires the operator to hand-type a Telegram message during its run is, by definition, a §11.4 PASS-bluff at the automation layer: it cannot run in CI, cannot catch regressions between manual runs, and the human dependency masks drift.

Three Herald live tests were previously NON-COMPLIANT for exactly that reason (now superseded — see §8 and HRD-143/144/145 marked Obsolete):

Legacy (manual) test

TestSubscribe_LiveBotAPI
tests/
test_wave6_live_loop.sh
Wave 6.5 lifecycle scenarios

Manual action it required

operator hand-sends a message during a 60s window
operator hand-sends a message; waits for the bot
reply
operator hand-sends each scenario's stimulus

The only autonomous way to DRIVE the conversation (post a message the system-under-test bot actually receives) is to act as a **real Telegram user**, which speaks **MTProto** — the same wire protocol Telegram's own mobile/desktop apps use. Herald uses the vendored Go library github.com/gotd/td (gotd/td) for this. The harness lives in `qa/herald/internal/mtproto/`.

The §11.4.98(B) carve-out makes this legitimate: a **one-time interactive credential bootstrap OUTSIDE test execution** is permitted (configuration, not test driving). You run `qa/herald mtproto login` **once** — Telegram sends a login code, you type it in — and every subsequent test invocation reuses the persisted session with zero human action. That one command is the entire purpose of this guide.

§1.3 What this enables once the session exists

- **E135** TestMTProto_Subscribe_AutonomousRoundTrip (HRD-140) — MTProto-driven replacement for TestSubscribe_LiveBotAPI.
- **E136** TestMTProto_Wave6_AutonomousClosedLoop (HRD-141) — full pherald → Claude Code → reply round-trip, replacing tests/test_wave6_live_loop.sh.
- **E137** TestMTProto_Wave65_LifecycleAutonomous (HRD-142) — Wave 6.5 fast-path lifecycle scenarios, replacing the --manual mode.
- **HRD-156** ATMOSphere integration — TestMTProto_ATMOSphere_SSoTChangeNotifiesGroup reads back the outbound notification's message_id via MTProto, proving the workable-items SSoT change actually reached the group (the outbound half is already LIVE-proven, evidence under docs/qa/HRD-156-LIVE-20260530T132303Z/).

§1.4 Where the code lives

File	Responsibility
qa/herald/cmd/ qa/herald/ mtproto_cmd.go	the qa/herald mtproto {login,whoami,logout} Cobra subcommand tree; env-var validation; the stdin code prompt + TTY check.
qa/herald/internal/ mtproto/mtproto.go	the Config struct, the Client interface (Connect/SendMessage/WaitForReply/WhoAmI/Close), the sentinel errors (ErrNoSession, ErrInvalidConfig, ErrSessionPasswordNeeded, ErrFloodWait), and Config.Validate.
qa/herald/internal/ mtproto/ client_live.go	the live gotd/td-backed liveClient implementation.
qa/herald/internal/ mtproto/session.go	session-file path resolution (~/.config/herald/mtproto.session, mode 0600), SessionExists, EnsureSessionDir.

File	Responsibility
qaherald/internal/ mtproto/errors.go	sanitizeMTPProtoError — redacts api_hash / session bytes / bot-token shapes from every error string (HRD-133 parity).
qaherald/internal/ lifecycle/ mtproto*_test.go	the integration_mtproto-tagged live tests (E135-E137 + HRD-156).

§2. Prerequisites + account-safety warnings (READ FIRST — ban risk)

⚠ **This is the single highest-risk operator path in Herald. A misconfigured MTPROTO login can get a Telegram account permanently banned.** Read this section in full before touching my . telegram . org. The rules below are cross-referenced against Telegram's official documentation at https://core.telegram.org/api/obtaining_api_id (verified 2026-05-31 — see the §11 Sources footer).

§2.1 The fundamental risk

Telegram's official docs state, verbatim:

"Due to excessive abuse of the Telegram API, all accounts that log in using unofficial Telegram API clients are automatically put under observation to avoid violations of the Terms of Service."

gotd/td is an unofficial client. The moment you log in with it, the QA account is **under observation**. This is normal and survivable — but it means the account-hygiene rules below are not optional politeness, they are ban-avoidance.

§2.2 The four hard rules

Rule 1 — Use a DEDICATED QA account, NOT the operator's primary identity (where practical). Two valid options, in this order of preference for a first cycle:

- **(a) Your personal Telegram account** — fastest; already has trust history with Telegram (lowers anti-abuse suspicion). The downside is that test-driver messages appear "from you" in the QA chat, and the account goes under observation. **Recommended for the first proof cycle**, then migrate to (b).
- **(b) A dedicated QA SIM / eSIM** — clean isolation; even if the QA account is banned your primary is untouched. The downside is a brand-new account has ZERO trust history → much higher ban risk in the first 30 days, so the recovery email (Rule 3) is mandatory before first login. **Recommended for production CI**, only after (a) has proven the harness works.

Rule 2 — NEVER use a VoIP / virtual number (Google Voice, Twilio, TextNow, etc.). Telegram's anti-abuse system flags virtual numbers aggressively and bans them with USER_DEACTIVATED (no appeal). Use only a real mobile number (option (a)) or a real SIM/eSIM (option (b)). This warning is cross-referenced against the gotd/td "How To Not Get Banned" guidance and Telegram's official abuse policy.

Rule 3 — Set up cloud 2FA + a recover@telegram.org recovery email BEFORE first login (see §4). Two reasons: (i) cloud 2FA protects the account if the api_hash or session file ever

leaks; (ii) pre-declaring the userbot's purpose to `recover@telegram.org` lets a human reviewer reverse an automated-system ban. Telegram's official docs confirm the recovery path:

"write to recover@telegram.org explaining what you intend to do with the API, asking to unban your account. Please note that emails are checked by a human, so automatically generated emails will be detected and banned."

A suggested (non-auto-generated) declaration template lives in `docs/requirements/blockers/missing_env_variables.md` §"CRITICAL".

Rule 4 — Treat `api_id`, `api_hash`, the 2FA password, and the session file as account-equivalent secrets. The `api_hash` + session file together are sufficient to impersonate the account. Never commit them, never paste them in chat, never share across projects (Telegram's terms are one-app-one-purpose; sharing risks a rate-limit ban on both projects). Herald's `.gitignore` excludes `.env` and the session never lives in-repo (it is written to `~/.config/herald/mtproto.session`, mode 0600).

§2.3 One phone = one `api_id`, forever

Telegram's official docs state:

"For the moment each number can only have one `api_id` connected to it."

If the phone you pick already has an app at `my.telegram.org/apps` from a previous project, **you MUST reuse that existing app** — there is no way to create a second one for the same number, and the `api_id/api_hash` are non-regenerable. Check first (§3.1).

§2.4 Passive-usage hygiene

Compose with §11.4.10 / §11.4.98:

- Use the harness **PASSIVELY** — receive more than you send.
- Honor `FLOOD_WAIT_<N>` responses (today they surface as a sanitized error you can pattern-match via `errors.As(*mtproto.FloodWaitError)`; the `gotd/contrib/floodwait` + `ratelimit` middlewares are a planned vendoring — see §10.5).
- Never flood, spam, scrape, fake subscribers, or fake view counters — Telegram's official docs list these as permanent-ban triggers:

"for flooding, spamming, faking subscriber and view counters" → permanent ban.

§2.5 What you need before starting

- A Telegram **user account** (option (a) or (b) above) that you control.
- The account must be a **member of the target chat** `HERALD_TGRAM_CHAT_ID` (e.g. the group -4946584787), or its driver messages won't reach the system-under-test bot. If you're using your personal account and you're already in that group, you're done; otherwise add the account to the group via the normal Telegram UI.
- A built `qaherald` binary: `go build -o /tmp/qaherald ./qaherald/cmd/qaherald`.
- An interactive terminal (a real TTY) for the one-time login — unless you deliberately use the delegated non-TTY flow (§9).

§3. Obtain api_id + api_hash at my.telegram.org/apps

Estimated time: ~5 minutes, one-time only.

§3.1 Check for an existing app FIRST

Because of the one-phone-one-api_id rule (§2.3), check before you create:

1. Open <https://my.telegram.org/auth> in a browser.
2. Enter the QA-driver phone in **E.164 format** (+countrycode + number, no spaces/dashes — e.g. +12025551234). This **MUST** be the same number you'll later put in HERALD_MTPROTO_PHONE.
3. Telegram sends a login code to that phone's Telegram **app** (an in-app notification, not an SMS). Enter it.
4. Navigate to <https://my.telegram.org/apps>.
5. **If a row already exists** ("App configuration" with an App api_id visible) — that IS your app. Reuse it. Copy the App api_id; retrieve the App api_hash via the edit/details view if your form version exposes it. Skip to §5.
6. **If no row exists** — proceed to §3.2.

§3.2 Create the application

1. On <https://my.telegram.org/apps>, click "**Create new application**".
2. Fill in the form. Telegram's validator is stricter than its hint text — the fields below carry the real, operator-confirmed constraints (recorded 2026-05-28 in the blocker doc):

Field	What to enter	The real constraint
App title	Herald, or HeraldQA, or Herald Test Harness	3-32 chars; letters + digits + spaces only. NO bare 2-letter all-caps acronyms (QA alone triggers Incorrect app name). Use a full word or a camelCase squish (HeraldQA).
Short name	heraldqa<random4> (e.g. heraldqa5kx9) — no underscores	5-32 chars; STRICTLY alphanumeric [a-zA-Z0-9] — underscores/dashes/spaces are REJECTED despite the hint; no leading digit; GLOBALLY UNIQUE across all Telegram apps (always append a random suffix on the first try).
URL	https://herald.local (or any valid http(s)://...)	NOT optional. Must carry a scheme; bare domains are rejected. A placeholder URL is fine — Telegram validates the shape only.
Platform	Desktop	If Other errors out on your form version, pick Desktop. Metadata only; doesn't gate API behaviour.
Description	Herald automation harness for closed-loop testing.	Plain ASCII only (the § / em-dash characters trip some validators). Under 200 chars. Avoid bare abbreviations.

3. Click "**Create application**".

4. Telegram displays two values:

- App `api_id` — a small integer (5-8 digits) → your `HERALD_MTPROTO_APP_ID`.
- App `api_hash` — a 32-character lowercase hex string → your `HERALD_MTPROTO_APP_HASH`.

5. **⚠️ Copy BOTH immediately into your password manager.** Telegram does NOT re-display the `api_hash` after you navigate away. Losing it means revoke + recreate, which invalidates any existing session.

§3.3 Shape sanity-check (matches `Config.Validate`)

`qaherald` validates these at runtime in `mtproto.Config.Validate` — match the shapes now to avoid a confusing error later:

- `HERALD_MTPROTO_APP_ID` — must parse as a **positive integer** (`AppID > 0`). A non-integer surfaces `HERALD_MTPROTO_APP_ID is not an integer`.
- `HERALD_MTPROTO_APP_HASH` — must be **exactly 32 lowercase-hex chars** (`[0-9a-f]{32}`). Wrong length surfaces `AppHash wrong length (want 32 hex chars)`; a non-hex char surfaces `AppHash contains non-hex character`.
- `HERALD_MTPROTO_PHONE` — must be **E.164**, starting with `+`. A missing `+` surfaces `Phone not in E.164 format (must start with +)`.

(The validator routes every message through `sanitizeMTPProtoError`, so the credential bytes themselves never appear in the error text.)

§3.4 Common mistakes

- Don't paste a **bot token** here — `my.telegram.org/apps` is for user-account apps. Bots use `@BotFather` (see [TELEGRAM.md](#)).
- Don't share the `api_hash` across projects (one-app-one-purpose; sharing risks a ban on both).
- Don't create apps in rapid succession — Telegram throttles app creation. Wait ≥ 1 hour between attempts.

§4. Set up cloud 2FA + recovery email on the QA account

Do this **before** first login (Rule 3, §2.2). It protects the account if a secret leaks, and the recovery email is your ban-reversal lifeline.

§4.1 Enable cloud Two-Step Verification (2FA)

In the Telegram **app** signed in as the QA account:

1. **Settings** → **Privacy and Security** → **Two-Step Verification**.
2. Tap **Set Password** / **Enable**. Choose a strong password — this is your **cloud 2FA password**, and it becomes `HERALD_MTPROTO_PASSWORD` (§5).
3. When prompted, set a **recovery email**. Use a mailbox you control. This is independent of the `recover@telegram.org` declaration (Rule 3) — the recovery email recovers a forgotten password; the `recover@telegram.org` email declares the userbot's purpose to reverse an anti-abuse ban.

4. Confirm the recovery email via the verification code Telegram sends to it.

If you choose to leave 2FA **disabled** (lower security; acceptable only for a throwaway QA account on a dedicated SIM), leave `HERALD_MTPROTO_PASSWORD` blank. The login flow handles a blank password cleanly — `qaherald` returns `auth.ErrPasswordNotProvided` internally so the code-only path short-circuits without prompting.

§4.2 Send the `recover@telegram.org` purpose declaration

Email `recover@telegram.org` from an address you control, in plain human language (auto-generated emails are detected and banned per Telegram's official docs), declaring what the userbot does. A ready-to-edit template lives in `docs/requirements/blockers/missing_env_variables.md` §"CRITICAL" — it describes the harness as sending a handful of automated test messages per run to a single group you own, receiving the system-under-test bot's replies, never messaging outside that group, and never flooding/spamming/scraping. Send this **first**; Telegram takes ~0-3 days to acknowledge, but you may proceed with §3-§6 in parallel.

§5. The `env vars` + `.env` placement

`qaherald mtpROTO` reads exactly **five** environment variables. Four feed `mtpROTO.Config`; the fifth (`HERALD_MTPROTO_ALLOW_NON_TTY`) is a login-flow override (§9).

Variable	Required?	What it is	Source
<code>HERALD_MTPROTO_APP_ID</code>	required	App <code>api_id</code> — a positive integer (5-8 digits)	<code>my.telegram.org/apps</code> (§3)
<code>HERALD_MTPROTO_APP_HASH</code>	required	App <code>api_hash</code> — 32-char lowercase hex	<code>my.telegram.org/apps</code> (§3)
<code>HERALD_MTPROTO_PHONE</code>	required	E.164 phone of the QA user account (+countrycode...)	the account you chose in §2.2
<code>HERALD_MTPROTO_PASSWORD</code>	required ONLY if 2FA on	cloud 2FA password; read from env only, never echoed	Telegram → Two-Step Verification (§4.1); blank if 2FA off
<code>HERALD_MTPROTO_ALLOW_NON_TTY</code>	optional	set to 1 to authorize the delegated non-TTY login flow (§9)	operator choice

§5.1 What each maps to in the code

All four `Config` values are assembled in `mtpROTOEnvConfig()` (`qaherald/cmd/qaherald/mtpROTO_cmd.go`). Missing/mis-shaped values surface a clear structured error up-front (no panic, no nil-deref) so a CI gate can grep the diagnostic:

- `HERALD_MTPROTO_APP_ID` unset → `HERALD_MTPROTO_APP_ID` is not set
– see `docs/requirements/blockers/missing_env_variables.md`

- `HERALD_MTPROTO_APP_HASH` unset → `HERALD_MTPROTO_APP_HASH` is not set
- `HERALD_MTPROTO_PHONE` unset → `HERALD_MTPROTO_PHONE` is not set
- `HERALD_MTPROTO_PASSWORD` is `OPTIONAL` — left blank when 2FA is disabled.

`HERALD_MTPROTO_PASSWORD` is read from the environment **ONLY** and is **never echoed at any verbosity level and never printed in an error message** — `sanitizeMTPROTOError` is the package-wide defense-in-depth backstop.

§5.2 Copy-pastable `.env` snippet

The project-local `.env` is git-ignored. Add:

```
# Herald — MTPROTO user-account harness (§11.4.98 full-
    automation; Wave 8 Track B)
HERALD_MTPROTO_APP_ID=<integer, 5-8 digits, from my.telegram.org/
    apps>
HERALD_MTPROTO_APP_HASH=<32-char lowercase hex, from
    my.telegram.org/apps>
HERALD_MTPROTO_PHONE=+12025551234          # E.164; the QA USER
    account, not a bot
HERALD_MTPROTO_PASSWORD=                    # only if 2FA
    enabled; otherwise leave blank

# Optional — authorize the delegated non-TTY login flow (see §9).
    Default unset.
# HERALD_MTPROTO_ALLOW_NON_TTY=1
```

Per `OPERATOR_CREDENTIALS.md` §"Resolution order (12-factor)", exported shell vars take precedence over `.env`. Put production secrets in your shell rc; put dev/test values in `.env`.

§5.3 The session file

After a successful login the session persists at:

```
~/ .config/herald/mtproto.session          (mode 0600, owner-only)
```

resolved by `mtproto.DefaultSessionFile()` against `$HOME`. The parent directory is created mode 0700 by `EnsureSessionDir()`. The session is **never committed** and **never echoed**; it is encrypted at rest only by the filesystem ACL, so the 0600/0700 permission discipline is the protection — do not loosen it. (You can override the path via `Config.SessionFile` in code; the CLI uses the default.)

§6. `qaherald mtproto login` — the one-time interactive bootstrap

This is the entire point of the guide: the **single command you run once**, the §11.4.98(B) permitted human-presence step. Everything after it is autonomous.

§6.1 The command sequence

```
# 1. Build the binary
go build -o /tmp/qaherald ./qaherald/cmd/qaherald

# 2. Ensure the four required env vars are exported (or present
    in .env)
#   HERALD_MTPROTO_APP_ID, HERALD_MTPROTO_APP_HASH,
#   HERALD_MTPROTO_PHONE,
#   HERALD_MTPROTO_PASSWORD (blank if 2FA off)

# 3. Run the one-time interactive login FROM A REAL TERMINAL (TTY
    required)
/tmp/qaherald mtpROTO login
```

§6.2 What you'll see (the real prompts)

The command emits, verbatim from `runMTPROTOLogin`:

```
qaherald mtpROTO login: connecting to Telegram (phone +12*****34)...
```

(The phone is masked — country-code + last two digits visible, the middle asterisk, via `maskPhone()`. Your full number is never echoed.)

Telegram then **sends a login code** to the QA account — an in-app push notification on the Telegram app (or SMS as a fallback). The CLI prompts for it:

```
Enter code Telegram sent to +12*****34:
```

Type the code and press Enter. The code is read as a single line from `stdin`, is short-lived (a few minutes), and is **never echoed back or logged**.

If the account has cloud 2FA enabled, the flow then submits `HERALD_MTPROTO_PASSWORD` **automatically** (from env — you are NOT prompted for it; the password never appears on screen). If 2FA is disabled and `HERALD_MTPROTO_PASSWORD` is blank, the flow skips the password step cleanly.

On success:

```
MTPROTO session active for @your_qa_username (user_id=123456789) — session
```

The session file is now written, mode 0600. **You never need to run login again** unless the session is invalidated (logout, password change, account suspend, or `AUTH_KEY_UNREGISTERED` — see §10).

§6.3 The TTY requirement (and the one override)

`login` **requires a real TTY by default** and refuses to run when `stdin` is piped/non-interactive. This is the anti-bluff guard: the login code **MUST** come from a human, never a hard-coded string. If `stdin` is not a TTY you get:

```
mtpROTO login: stdin is not a TTY — the one-time interactive bootstrap req
```

The single override is `HERALD_MTPROTO_ALLOW_NON_TTY=1` — see §9 for when it's permitted and how it works.

§6.4 What login refuses to do

- **It never creates a new Telegram account.** If Telegram asks the flow to sign up (the number is unregistered), login refuses with: `mtpROTO login: refusing to create a new Telegram account — pre-register the QA user account at https://my.telegram.org first`. Register the account in the Telegram app first.
 - **It never hard-codes a code.** The human-presence check is enforced by Telegram itself (the out-of-band SMS/app-push delivery), not just by the local TTY check.
-

§7. whoami to verify + logout to tear down

§7.1 qaherald mtpROTO whoami — verify the session is alive

`whoami` connects with the persisted session and prints the authenticated identity. It is **fully autonomous** — **it never prompts**. Use it as a §107 anti-bluff sanity check before launching a long QA campaign (it confirms Telegram hasn't invalidated the session via account-suspend / password-change / server-side logout).

```
/tmp/qaherald mtpROTO whoami
```

Success:

```
MTPROTO session OK: user_id=123456789 username=@your_qa_username session=
```

If the session file is missing, `whoami` fails fast **without a network call** and points you at login:

```
mtpROTO whoami: mtpROTO: no session — run `qaherald mtpROTO login` first
```

(An account with no @username set prints `username=(no username)` — that's fine; the tests match by `user_id`, not `username`.)

§7.2 qaherald mtpROTO logout — invalidate + remove

Logout sends `tg.AuthLogOut` to invalidate the session **server-side**, then removes the local session file. Use it before rotating credentials, tearing down a QA bench, or switching QA accounts.

```
/tmp/qaherald mtpROTO logout
```

Success:

```
mtpROTO logout: server-side LogOut OK  
mtpROTO logout: removed local session /Users/you/.config/herald/mtpROTO.se
```

If no local session exists, logout is a clean no-op:

```
mtpROTO logout: no local session at /Users/you/.config/herald/mtpROTO.sess
```

The local file is removed even if the server-side LogOut fails, so disk state stays consistent. After logout you must run `login` again to re-bootstrap.

§8. Running the autonomous live tests (E135-E137 / HRD-156)

Once the session exists (`whoami` is green), the live tests are **fully autonomous and re-runnable endlessly** — no human action.

§8.1 Prerequisites for the live tests

The tests are gated behind the Go build tag `integration_mtproto` and `t.Skip` (per §11.4.3) unless ALL of these are present:

- `HERALD_MTPROTO_APP_ID` / `HERALD_MTPROTO_APP_HASH` / `HERALD_MTPROTO_PHONE` (+ `HERALD_MTPROTO_PASSWORD` if 2FA).
- `HERALD_TGRAM_BOT_TOKEN` + `HERALD_TGRAM_CHAT_ID` (the system-under-test bot + the shared group — see [TELEGRAM.md](#)).
- A persisted MTProto session (`~/config/herald/mtproto.session` present — i.e. you ran `qaherald mtproto login`). If it's missing, each test SKIPS with `MTProto session file missing — run 'qaherald mtproto login' first`.
- For E136, `HERALD_CLAUDE_BIN` (or `claude` on `PATH`) — the Claude Code dispatcher half of the round-trip.

§8.2 The commands (verbatim from `scripts/e2e_bluff_hunt.sh`)

```
# E135 — MTProto-driven Subscribe replacement (HRD-140)
go test -tags=integration_mtproto -count=1 -short -timeout=180s \
  -run 'TestMTProto_Subscribe_AutonomousRoundTrip' ./qaherald/
  internal/lifecycle/...

# E136 — full pherald→Claude Code→reply round-trip via MTProto
(HRD-141)
go test -tags=integration_mtproto -count=1 -short -timeout=420s \
  -run 'TestMTProto_Wave6_AutonomousClosedLoop' ./qaherald/
  internal/lifecycle/...

# E137 — Wave 6.5 fast-path lifecycle scenarios via MTProto
(HRD-142)
go test -tags=integration_mtproto -count=1 -short -timeout=300s \
  -run 'TestMTProto_Wave65_LifecycleAutonomous' ./qaherald/
  internal/lifecycle/...

# HRD-156 — ATMOSphere SSoT-change → group-notify, message_id
  read-back via MTProto
go test -tags=integration_mtproto -count=1 -short -timeout=300s \
  -run 'TestMTProto_ATMOSphere_SSoTChangeNotifiesGroup' ./
  qaherald/internal/lifecycle/...
```

These also run as invariants **E135, E136, E137** inside `scripts/e2e_bluff_hunt.sh` — when the session + creds are present they convert from SKIP-with-reason to live PASS.

§8.3 Where evidence lands (§107.x)

Per the docs/qa evidence mandate, every live run captures its bidirectional transcript under `docs/qa/<run-id>/`. Representative existing artefacts:

- `docs/qa/HRD-156-LIVE-20260530T132303Z/` — the LIVE-proven ATMOSPHERE outbound (`message_id` read-back).
- `docs/qa/HRD-LIVE-MTPROTO-20260528T125321Z/` — the bot-to-bot-wall empirical proof (§1.1).
- `docs/qa/HRD-LIVE-20260528T082128Z/` — the §11.4.98 audit anchor.

When you run a fresh campaign, create `docs/qa/HRD-<NNN>-<TS>Z/` (timestamp UTC) and let the harness journal the round-trip into it. A live PASS with no committed transcript is itself a §107 PASS-bluff.

§8.4 The §11.4.98 contract this satisfies

Running `qaherald mtpROTO login once` (§6) is the permitted one-time bootstrap; the E135-E137 + HRD-156 runs above are then fully self-driving. This is what moved the three legacy tests from NON-COMPLIANT to COMPLIANT and let HRD-143/144/145 be marked Obsolete (superseded-by-design-change, per §11.4.90).

§9. The non-TTY delegated flow (`HERALD_MTPROTO_ALLOW_NON_TTY=1`)

`login` requires a TTY by default (§6.3). The single, explicit override is `HERALD_MTPROTO_ALLOW_NON_TTY=1`.

§9.1 What it permits — and what it does NOT

When set, the override authorizes **non-TTY piping of the login process ONLY**. It does NOT weaken the human-presence requirement:

- The operator (a human) STILL types the 5-digit code Telegram sent to their phone — the code is delivered to the running `login` process via a FIFO/pipe driven by the conductor (e.g. the conductor prompts the operator over chat, the operator replies with the code, the conductor writes it into the FIFO).
- **Hard-coded codes remain prohibited.** The human-presence guarantee is enforced by Telegram itself via the out-of-band SMS/app-push delivery — the code is short-lived and only the real account-holder receives it — not by the local TTY check. The override simply turns off the local TTY assertion so the code can arrive through a pipe instead of a keyboard.

§9.2 When §11.4.98(B) permits it

§11.4.98(B) permits a **one-time interactive credential bootstrap OUTSIDE test execution** (configuration, not test driving). The delegated flow is still that same one-time human-presence

bootstrap — the human still supplies a code only they could have received — so it stays within the carve-out. It is appropriate when:

- the bootstrap must run on a headless host (no interactive shell), and
- a conductor process can relay the operator-typed code into the FIFO.

It is NOT a license to automate the code: a hard-coded or programmatically-guessed code is a §11.4.98 PASS-bluff regardless of this flag.

§9.3 How to use it

```
# Authorize the non-TTY path (in .env or exported)
export HERALD_MTPROTO_ALLOW_NON_TTY=1

# Drive login with the operator-typed code arriving via a FIFO
#   the conductor writes:
mkfifo /tmp/mtproto-code.fifo
/tmp/qaherald mtproto login < /tmp/mtproto-code.fifo &
# ... conductor asks the operator for the code over chat, then:
printf '%s\n' "<the code the operator received>" > /tmp/mtproto-
code.fifo
```

A turnkey conductor script for this is planned (see the Continuation note); today the FIFO wiring is manual.

§10. Troubleshooting cookbook

Each entry names the **real error string** (from the code or the gotd/td / Telegram layer) and the fix.

§10.1 PHONE_NUMBER_INVALID

Symptom: login fails immediately with a Telegram PHONE_NUMBER_INVALID error (often wrapped as mtproto login: auth: ...).

Root cause: HERALD_MTPROTO_PHONE is not valid E.164, or is not a registered Telegram user account, or carries spaces/dashes.

Fix: set HERALD_MTPROTO_PHONE to strict E.164 — +countrycode + digits, no spaces/dashes (e.g. +12025551234). Config.Validate catches a missing + early with Phone not in E.164 format (must start with +). Confirm the number is a real user account registered in the Telegram app.

§10.2 PHONE_CODE_INVALID

Symptom: after you type the code, login fails with PHONE_CODE_INVALID.

Root cause: wrong code, a stale/expired code (they live only a few minutes), or a typo with leading/trailing whitespace.

Fix: re-run `qaherald mtpROTO login` and enter the **fresh**est code Telegram sent (Telegram invalidates older codes when it sends a new one). Don't reuse a code from a previous attempt. If you submit an empty line you'll get `empty code` — re-run `\qaherald mtpROTO login``.

§10.3 SESSION_PASSWORD_NEEDED / 2FA

Symptom: login stalls or errors at the password step; or `mtpROTO: SESSION_PASSWORD_NEEDED` — set `HERALD_MTPROTO_PASSWORD`.

Root cause: the account has cloud 2FA enabled but `HERALD_MTPROTO_PASSWORD` is blank.

Fix: set `HERALD_MTPROTO_PASSWORD` to the account's cloud 2FA password (Telegram → Settings → Privacy and Security → Two-Step Verification — see §4.1). The password is read from env only and never echoed. Re-run login.

§10.4 AUTH_KEY_UNREGISTERED / session invalidated

Symptom: `whoami` or a live test that previously worked now fails; or `mtpROTO: no session` — run `\qaherald mtpROTO login` first`` after the session was working.

Root cause: Telegram invalidated the session server-side — you (or another device) logged the account out, changed the 2FA password, the account was suspended, or you ran `qaherald mtpROTO logout`.

Fix: re-run `qaherald mtpROTO login` to re-bootstrap. If the account was suspended/banned, email `recover@telegram.org` (§4.2) before re-attempting — repeated failed logins on a flagged account compound the problem.

§10.5 FLOOD_WAIT_<N>

Symptom: a send or poll returns `mtpROTO: FLOOD_WAIT_<duration>` (e.g. `mtpROTO: FLOOD_WAIT_30s`).

Root cause: Telegram's server-side rate-limiter triggered — too many actions too fast.

Fix: honor the retry-after. The duration is carried on `mtpROTO.FloodWaitError`; pattern-match with `errors.As(err, &fw)` against `*mtpROTO.FloodWaitError` (or `errors.Is(err, mtpROTO.ErrFloodWait)`) and sleep `fw.RetryAfter` before retrying. Reduce send concurrency; use the harness passively (§2.4). The `gotd/contrib floodwait + ratelimit` middlewares are NOT yet vendored under `submodules/gotd-td/` — when they land they'll be registered via `telegram.Options.Middlewares` and back off automatically; until then the error surfaces for the caller to handle.

§10.6 stdin is not a TTY (non-TTY refusal)

Symptom: `mtpROTO login: stdin is not a TTY` — the one-time interactive bootstrap requires a human operator to enter the Telegram login code....

Root cause: login was run with piped/redirected stdin (CI, `nohup`, a non-interactive harness) without the override.

Fix: run `login` from a real interactive shell, OR — only if you genuinely need a headless delegated bootstrap — set `HERALD_MTPROTO_ALLOW_NON_TTY=1` and drive the code through a FIFO (§9). Never hard-code the code.

§10.7 refusing to create a new Telegram account

Symptom: `mtproto login`: refusing to create a new Telegram account — pre-register the QA user account at <https://my.telegram.org> first.

Root cause: the number in `HERALD_MTPROTO_PHONE` is not a registered Telegram user — Telegram's flow tried to route to sign-up, which `qaherald` refuses (it never creates accounts).

Fix: register the account in the official Telegram app first (install Telegram, sign up with that number), then re-run `login`.

§10.8 Incorrect app name at my.telegram.org

Symptom: creating the app at `my.telegram.org/apps` returns `Incorrect app name`.

Root cause: despite the wording, the usual culprit is the **Short name** field (Telegram internally calls it the app name): it must be 5-32 STRICTLY alphanumeric chars — **no underscores/dashes**, no leading digit, globally unique. The **App title** can also trigger it if it contains a bare all-caps acronym (QA).

Fix: use a Short name like `heraldqa5kx9` (append a random 4-char suffix; no underscores) and an App title like `Herald` or `HeraldQA` (§3.2). If all combinations still fail, the account may be rate-limited — wait 1 hour.

§10.9 chat not found / driver message never reaches the bot

Symptom: a live test runs but the system-under-test bot never sees the driver message; or `resolvePeer: chat <id> not found in dialog list (user not a member?)`.

Root cause: the QA user account is NOT a member of `HERALD_TGRAM_CHAT_ID`, or the chat id is wrong-format.

Fix: ensure the QA account is a member of the target group (§2.5). Verify the chat id format (positive = DM; `-100...` = supergroup/channel — see [TELEGRAM.md](#) §6). For supergroups the `-100` prefix is part of the canonical id — do not strip it.

§10.10 `USER_DEACTIVATED` / account banned

Symptom: every `MTPROTO` call fails with `USER_DEACTIVATED` (or login is rejected outright).

Root cause: Telegram's anti-abuse system banned the account — most commonly because it used a VoIP/virtual number (§2.2 Rule 2), flooded/spammed, or a brand-new account hit heavy automation before its purpose was declared.

Fix: there is no self-service un-ban. Email recover@telegram.org from an address you control, in plain human language, declaring the userbot's legitimate purpose (§4.2). A human reviewer can reverse an automated ban when the use case is clearly legitimate. For future

accounts, follow all four hard rules in §2.2 — especially: never use a VoIP number, and pre-declare via `recover@telegram.org` before first heavy use.

§11. References

§11.1 Source files (the authority for every command, prompt, and error string)

- `qaherald/cmd/qaherald/mtproto_cmd.go` — the `qaherald mtproto {login,whoami,logout}` subcommand tree; `mtprotoEnvConfig()`; the stdin code prompt (`loginAuthenticator.Code`); `maskPhone`; `isStdinTTY`; the non-TTY refusal + `HERALD_MTPROTO_ALLOW_NON_TTY` override.
- `qaherald/internal/mtproto/mtproto.go` — `Config`, the `Client` interface, `Config.Validate`, and the sentinel errors (`ErrNoSession`, `ErrInvalidConfig`, `ErrSessionPasswordNeeded`, `ErrFloodWait`, `FloodWaitError`).
- `qaherald/internal/mtproto/client_live.go` — the live `gotd/td`-backed `liveClient` (`Connect/SendMessage/WaitForReply/WhoAmI/Close`, `resolvePeer`).
- `qaherald/internal/mtproto/session.go` — `DefaultSessionFile` (`~/ .config/herald/mtproto.session`), `ResolvedSessionFile`, `SessionExists`, `EnsureSessionDir`.
- `qaherald/internal/mtproto/errors.go` — `sanitizeMTPROTOError` (HRD-133 parity — credential-shape redaction).
- `qaherald/internal/lifecycle/mtproto_subscribe_test.go` / `mtproto_wave6_loop_test.go` / `mtproto_wave65_lifecycle_test.go` / `mtproto_atmosphere_test.go` — the `integration_mtproto`-tagged live tests E135-E137 + HRD-156.

§11.2 HRDs

HRD	Description	Status
HRD-133	MTPROTO error sanitization (credential-shape redaction) — <code>sanitizeMTPROTOError</code> parity.	LIVE
HRD-139	Wave 8 Track B — <code>qaherald/internal/mtproto/</code> MTPROTO user-account harness scaffold.	LIVE (E134)
HRD-140	Test 1 — <code>TestMTPROTO_Subscribe_AutonomousRoundTrip</code> (E135).	live-evidence-gated
HRD-141	Test 2 — <code>TestMTPROTO_Wave6_AutonomousClosedLoop</code> (E136).	live-evidence-gated
HRD-142	Test 3 — <code>TestMTPROTO_Wave65_LifecycleAutonomous</code> (E137).	live-evidence-gated
HRD-143/144/145	The three legacy manual tests this harness replaces — marked Obsolete (superseded-by-design-change, §11.4.90).	Obsolete
HRD-156	ATMOSPHERE integration WS-5 — anti-bluff full-automation; SSoT-change → group-notify with <code>message_id</code> read-back via MTPROTO.	open (outbound LIVE-proven)

§11.3 Vendored library

- github.com/gotd/td (gotd/td) — the Go MTPROTO Telegram client Herald uses for the user-account harness; vendored under `submodules/gotd-td/`. Read its "How To Not Get Banned" guidance before heavy use. The planned github.com/gotd/contrib floodwait + ratelimit middlewares are not yet vendored (§10.5).

§11.4 Sibling guides + spec

- [TELEGRAM.md](#) — the **bot** side (BotFather walkthrough, HERALD_TGRAM_BOT_TOKEN/HERALD_TGRAM_CHAT_ID, chat-type taxonomy, the Bot API caps). This MTPROTO guide is its user-account companion.
- [OPERATOR_CREDENTIALS.md](#) — umbrella credentials guide; the `.env` resolution order; the audit checklist.
- [MESSENGER_CHANNELS.md](#) — the multi-channel framework.
- `docs/requirements/blockers/missing_env_variables.md` — the original operator blocker with the `my.telegram.org` form-validation gotchas, the `recover@telegram.org` declaration template, and the phone-choice trade-off table.
- `docs/specs/mvp/specification.V4.md` — §32 inbound contract; §43 channel + flavor catalogue.

§11.5 Constitutional anchors

- HelixConstitution §11.4.98 — Full-Automation Anti-Bluff Mandate (the authority for the autonomous-test requirement + the §(B) one-time-bootstrap carve-out).
- HelixConstitution §11.4.99 — Latest-Source Documentation Cross-Reference Mandate (the authority for this guide's `Sources verified` footer; the anchoring case study is literally an MTPROTO setup guide).
- HelixConstitution §11.4.83 → Herald §107.x — `docs/qa/<run-id>/evidence` mandate.
- HelixConstitution §11.4.90 — Obsolete status (HRD-143/144/145).
- Herald §107 — end-user-usability covenant.

Sources verified 2026-05-31

Per HelixConstitution §11.4.99 (Latest-Source Documentation Cross-Reference Mandate). Telegram is a **risk-classified service** (§11.4.99(D)) — a misconfigured MTPROTO login can permanently ban the operator's account, so every account-safety instruction below was cross-referenced against the LATEST official online documentation before publication.

Source	URL / path	Covers
Telegram official — Obtaining <code>api_id</code>	https://core.telegram.org/api/obtaining_api_id	§2 (verbatim "all accounts that log in using unofficial Telegram API clients are automatically put under observation"; "for flooding, spamming, faking subscriber and view counters" → permanent ban; <code>recover@telegram.org</code> un-ban path checked by a human); §2.3 (verbatim "each number can only have one <code>api_id</code> connected to it"); §3 (the

Source	URL / path	Covers
gotd/td library (Go MTProto client)	https://github.com/gotd/td (vendored submodules/gotd-td/)	obtain-api_id steps: official-app signup → my.telegram.org → API development tools → api_id/api_hash). §1.2 (the library Herald uses for the user-account driver); §2.2/§2.4 (the "How To Not Get Banned" guidance + floodwait/ratelimit middleware references); §6 (the auth flow: auth.NewFlow + UserAuthenticator code/password + pluggable session storage).
Herald source — qaherald mtproto	qaherald/cmd/qaherald/mtproto_cmd.go + qaherald/internal/mtproto/*.go (repo HEAD 2026-05-31)	§3.3, §5, §6, §7, §9, §10 — every env-var name, command, prompt, output line, and error string is quoted verbatim from this source, not invented.
Herald operator blocker (prior cross-reference, 2026-05-28)	docs/requirements/blockers/missing_env_variables.md	§2 (the four hard rules + VoIP-ban warning + recover@telegram.org template); §3.2 (the my.telegram.org form-validation gotchas — operator-confirmed Incorrect app name cases).
Herald empirical QA evidence	docs/qa/HRD-LIVE-MTPROTO-20260528T125321Z/, docs/qa/HRD-156-LIVE-20260530T132303Z/, docs/qa/HRD-LIVE-20260528T082128Z/	§1.1 (bot-to-bot wall proof); §8.3 (live-evidence artefact paths).

Re-verification cadence (§11.4.99(C)): Telegram is risk-classified → **90-day max staleness**, next re-verification due **2026-08-29**. Re-verify earlier on: a Telegram API breaking-change announcement, a gotd/td major-version bump, an operator-reported error whose string doesn't match this cookbook, or a Herald vN.0.0 release boundary.

Negative findings (honest gaps): Telegram's official obtaining_api_id page does NOT itself address api_hash-sharing, VoIP/virtual-number bans, or the cloud-2FA recommendation in the excerpt fetched — those §2 warnings derive from the gotd/td "How To Not Get Banned" guidance + Herald's own empirical operator testing (recorded in the blocker doc), and are flagged as such rather than misattributed to the official page. The submodules/gotd-td/ working tree was not checked out at authoring time, so the gotd/td guidance was cross-referenced via its canonical GitHub source.

End of guide.